Project1 1 Report

ShiqiXu

Directory

1. Copy 1
1.1. Requirement 1
1.2. Usage 1
1.3. Tricks 2
1.3.1. Clock( ) function 1
1.3.2.long getFilesize(constant char*filename) 2
1.4. Some reviews 2
1.5. Statistics and analysis 2
2. Shell 3
2.1. Requirement 3
2.2.Usage 3
2.3.Tricks 3
2.3.1. Parseline 4
2.3.2.Execute commands 4
2.3.3.Nested fork 4
2.3.4. Pipe create 5
2.4. Some reviews 6
2.4.1. Blocked pipes 6
2.4.1.1.close pipe early 6
2.4.1.2.pipes are not shut down in a timely manner 6
3. Matrix 7
3.1. Requirement 7
3.2.Usage 7
3.3.Tricks 8
3.3.1. Matrix partition 8
3.3.2.Multi-threads matrix multiplication 8
3.4. Statistics and analysis 9

1. Copy

1.1. Requirement

Forking two processes to copy file using pipe system call.The name of two files are given as command line arguments

1.2. Usage

```
./copy <InputFile><OutputFile><BufferSize>
```

This command can copy content from InputFile to OutputFile

You should provide the path of InputFile and OutputFile, and the buffer size however, the path of OutputFile can be a directory, and the file itself does not exist, so it will be created.

1.3. Tricks

1.3.1. Clock( ) function

Clock( ) function : It can return the CPU ticks not real time;

```
Clock_t Start,End;
//...
Start=clock( );
ForkPID =fork( );//forking another process...
End=clock();// at the end of father process
elapsed=((double)(end-start))/CLOCKS_PER_SEC*1000;
//In this way, we can get the whole approximate time by clock function
```

1.3.2. long getfilesize (const char *filename)

getfilesize( ) function can return the size of file ,witch can be used to conduct statistics, its usage can cause CPU file resource allocation change, so it can affect the clock( ) if program runs in two different process.

1.4. Some reviews

1.4.1.1. Clock time error about forking process

At the very beginning, I use clock( ) function by mistake like this:

```
Clock_t Start,End;
//...
ForkPID =fork( );//forking another process...
Start=clock( );//at the start of son process
End=clock();// at the end of father process
elapsed=((double)(end-start))/CLOCKS_PER_SEC*1000;
```

Unfortunately, the result is very unusual like -324343545546546 ms, after carefully check, I find I use fork by mistake, the two processes may using different CPU resource ,and the clock( ) function are using the same CPU ticks to calculate time.However the two process are using different CPU,so the End and Start cannot be used to calculate elapsed.

1.5. Statistics and analysis

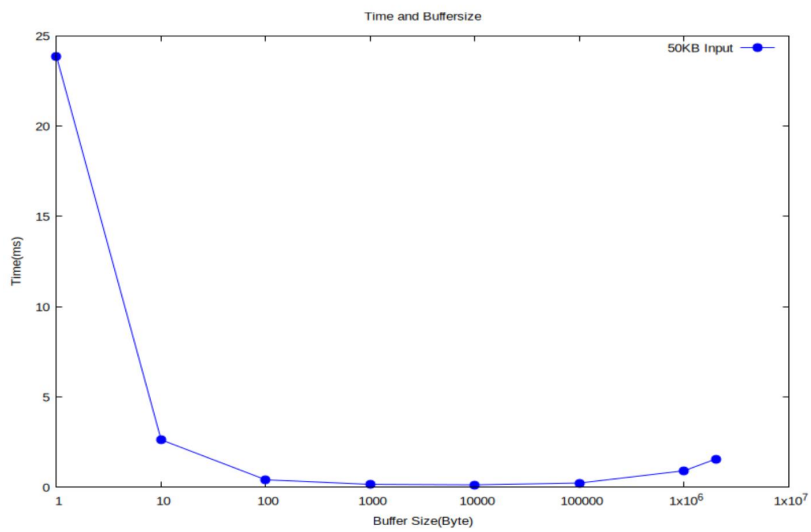

Figure1: time and Buffersize

From figure 1, we can see that as the increasing of buffersize, the consumed time is declining ,The place where the descent is most drastic is during 1byte and 10bytes, and Then keep it in a smooth state,however after 100000bytes of buffersize,There is an upward trend.

Larger buffersize can avoid constantly access files and output into files, so in the very beginning ,the effect is significant.

After seeking related information, I draw the conclusion that its reason is CPU cache.(Type in command below can check the CPU information: 'lscpu')

My vmmware cpu's L1cache is 64KiB,this number is approximately at the beginning of an uptrend, so it lets me think the relation of them.because when I choose the buffersize less than 64KiB, the cache can hold the buffersize, and other buffer files, so there is no cache miss.However, when the number is more than 64KiB, the cache cannot handle even just buffersize, resulting in the increasing of cache miss,witch will increase cache content adjustment time.

So choose the buffersize which matches the cpu cache is best.for my device,choose 64KiB is best.

2. Shell

2.1. Requirement

Write a shell-like program that illustrates how Linux spawns processes. It should handle the commands with arguments and the commands connected by pips. Server program is needed to return command result to client request.

2.2. Usage

```
./shell <Port>
```

This command will create a server with the given port, then we can connect the server by the following code:

```
telnet localhost <Port>
```

2.3. Tricks

2.3.1. Parseline

parseline( ) function can be used to parse command provided by client, It can read command string, and divide it into several parts and save them into command_array[]. The operation will get blocked if it reads '|' ,witch are called 'pipe character' .Its code implementation is as follows

```
int parseLine(char*line,char*command_array[]){
        char*p;
        int count=0;
        p=strtok(line," ");
        while(p&&strcmp(p,"|")!=0){
                command_array[count]=p;
                count++;
                p=strtok(NULL," ");
        }
        command_array[count]=NULL;
        return count;
}
```

2.3.2. Execute commands

```
if(execvp(command_array[0],command_array)==-1){
                printf("ERROR:running command:'%s'\n",buffer);
                exit(0);
```

2.3.3. Nested fork

I use two nested fork to complete the whole server program .

the first layer of fork .

Its function is to execute an infinite loop, continuously waiting in the parent process to receive connection requests from different client sides, and at the same time, creating multiple child processes that correspond one-to-one with different clients to handle the requests.

```
while(1){
        client_sockfd=accept(sockfd,(sockaddr*)&client_addr,&len);
        pid_t pid=fork();
```

he second layer of fork.

Its function is to continuously create child processes through a while loop to receive the information from the previous pipeline, execute commands, redirect the output and pass it to the next pipeline until all the strings are processed, and then output the result to the client side.

```
while(line!=NULL){
                parseLine(line,command_array);
                line=strtok(NULL,"");//this has to be none string
                count++;
        pid_t pid_third=fork();
```

```
char*line;
        char buffer_temp[1024];
        int total=0;
        strcpy(buffer_temp,buffer);
        line=buffer_temp;
        while(line!=NULL){
                parseLine(line,command_array);
                line=strtok(NULL,"");
                total++;
        }
        total-=1;
        int count=0;
        line=buffer;
        int mypip[total][2];
```

Second , create pip.

```
for(int i=0;i<total;i++){
                if(pipe(mypip[i])==-1){
                        printf("ERROR:piping\n");
                        exit(2);
                }
        }
```

Third , Use pipelines to pass information

```
if(line==NULL){
        close(mypip[count-2][1]);
        close(0);
        dup2(mypip[count-2][0],STDIN_FILENO);
        dup2(client_sockfd,STDOUT_FILENO);
        close(mypip[count-2][0]);
}
else{
if(count==1){
        close(mypip[count-1][0]);
        close(1);
        dup2(mypip[count-1][1],STDOUT_FILENO);
        close(mypip[count-1][1]);
}
else{
        close(mypip[count-2][1]);
close(0);
dup2(mypip[count-2][0],STDIN_FILENO);
close(1);
close(mypip[count-1][0]);
dup2(mypip[count-1][1],STDOUT_FILENO);
close(mypip[count-2][0]);
close(mypip[count-1][1]);//close pip timely,in case of pip blocking
}
}
```

2.4. Some reviews

2.4.1. Blocked pipes

2.4.1.1. close pipe early.

This segmentation of code is running in father process ,it is used to close previous unused pipe.

```
if(pid_third>0){
//cannot close father pipe early,else pip cannot be used,if do not close it ,the pip is blocked untill exit
                    if(count>1){
                              close(mypip[count-2][0]);
                              close(mypip[count-2][1]);//close father pip
                    }
          }
```

At first, I write 'close(mypip[count-1][0]', resulting in an early shutdown of the pipeline.

2.4.1.2. Pipes were not shut down in a timely manner

```
else{
                              close(mypip[count-2][1]);
                    close(0);
                    dup2(mypip[count-2][0],STDIN_FILENO);
                    close(1);
                    close(mypip[count-1][0]);
                    dup2(mypip[count-1][1],STDOUT_FILENO);
                    close(mypip[count-2][0]);
                    close(mypip[count-1][1]);//close pip timely,in case of pip blocking
                    }
```

At first, I did not write the last two lines of code , that can cause previous pipe cannot close the pipe sides,resulting in the following process cannot read information from the specified pipe,because pipe can only be used in one direction.

3. Matrix

3.1. Requirement

implement matrix multiplication using matrix partition based algorithm with time complexity O(n3) in two ways :single process, multiple processes.

3.2. Usage

```
./single #read from data.in file, write to data.out file,run in single process
./multi #read from data.in file, write to data.out file,run in multi process
./multi <Size> #generate Size*Size matrix A and B, do A*B and write to random.out file
```

3.3. Tricks

3.3.1. Matrix partition

```
void matrix_multiply2(int matrix_1[400][400],int matrix_2[400][400],int result[400][400],int N) {
    for (int i = 0; i < N; i += B) {
        for (int j = 0; j < N; j += B) {
            for (int k = 0; k < N; k += B) {
                // multiply every small matrix
                for (int ii = i; ii < i + B && ii < N; ii++) {
                    for (int jj = j; jj < j + B && jj < N; jj++) {
                        int sum = 0;
                        for (int kk = k; kk < k + B && kk < N; kk++) {
                            sum += matrix_1[ii][kk] * matrix_2[kk][jj];
                        }
                        result[ii][jj] += sum;
                    }
                }
            }
        }
    }
}
```

The matrix is partitioned into a number of B*B size matrix ,then multiply them in a algorithm based matrix partition multiply:

3.3.2. Multi-threads matrix multiplication

I create several threads to process some certain lines of result matrix, for example,I have two 5*5 matrix, so I use three threads, the two for line1,2 and line 3,4 ,the last thread is for the last line of result matrix, so it is a rough matrix partition, but it works, code is like this:

```
void*phread_multi(void*ranknum){
        long begin=(long)ranknum;//long type can avoid precison loss
        long begin_num=begin*process_size;
        long last_num=(begin+1)*process_size-1;
        if(last_num>size-1)//take attention for this comparison
        {
                last_num=size-1;
        }
        for(int i=begin_num;i<=last_num;i++){
                for(int j=0;j<size;j++){
                        for(int k=0;k<size;k++){
                                result[i][j]+=matrix_1[i][k]*matrix_2[k][j];
                        }
                }
        }
        pthread_exit(NULL);//finish this thread

}
```

3.4. Statistics and analysis

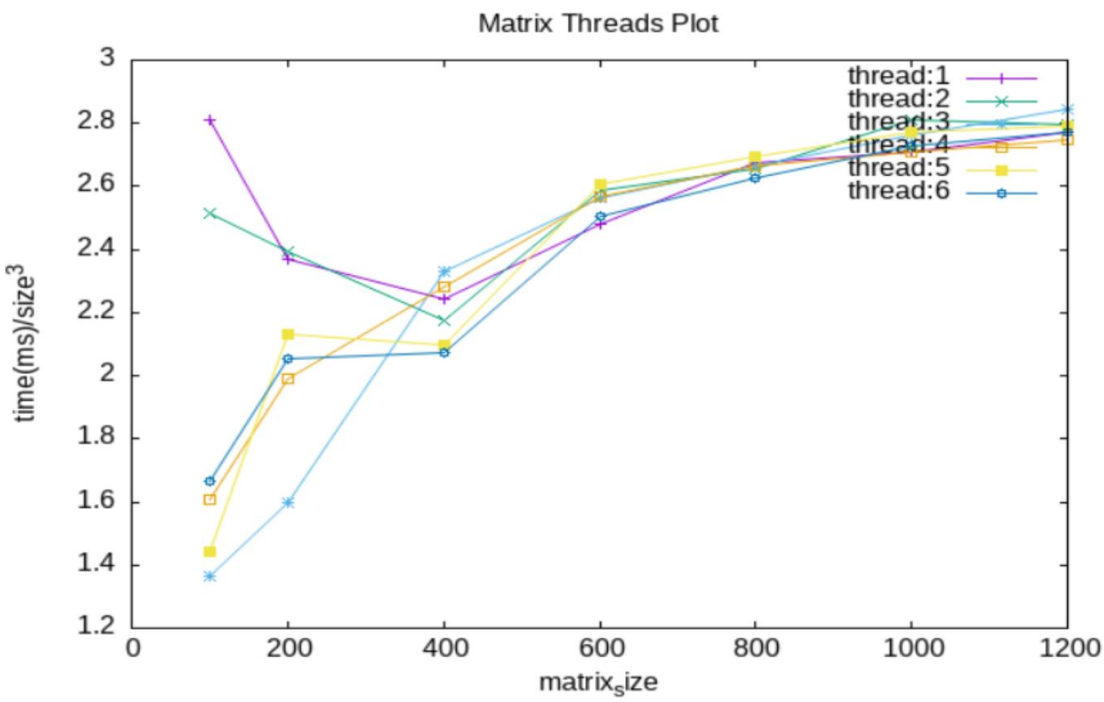

Figure2:matrix threads plot

Multi.c files can produce required size of matrix, so; I use a shell script to generate a series of size of matrix continuously, the plot result is to be seen in figure2.

To make ultimate plot is easy to observe, I divide the time by 3 times of matrixsize,(because the algorithm complexity is O(n3)).

From figure2, before the matrixsize is 400, thread 1 and 2 perform the worst and thread 3 perform best, thread 456 are almost the same, but after 400, all the thread go into the same trend,

I think the reason are cpu cores and threads per core. Use command 'lscpu' can get cores and threads information.. My VMware has 2 CPU ,and 1 thread per core. When I check my algorithm ,I find the reason.look this code.

```
process_size=(size%threadnum==0?size/threadnum:(size/threadnum)+1);
```

This algorithm allocates tasks by lines,such as when the size is 9*9 using 2 threads to process it, then the two threads are allocated distinct lines : 4 and 5. so the total runtime is up to the one which processes the 5 lines. However, if I use 3 threads, I can equally allocate them 3 lines, so thay can work more efficiently. So that's why 2 threads in two cores system perform out of expect.

What the plot isn't show is that when matrixsize is less than 10,thread 1 perform the best. Other perform worse as the threads increasing. That is because when matrixsize is small, threads switching cost the main time.

Anyway, compare to 1 threads in large matrixsize, multi-threads perform better than 1threads.

So if I change my algorithm, to make each threads get same tasks, the threads 2 may perform the best .